

Credit Card Fraud Detection Final Report

Course: CS 4662-01 - ADV MACHINE & DEEP LEARNING

Instructor: Mohammad Pourhomayoun

Team Members: Alyssa Tu, Humayra Rashid, John Lopez, Brian Andrade, Slok Patel

1. Project Description, Details, Goals

In this project, we developed and evaluated multiple machine learning models to detect fraudulent credit card transactions. The dataset represents real-world European credit card transactions and is highly imbalanced, with fraud cases accounting for less than 0.2% of all records. The main challenge is building models that can accurately detect rare fraud cases without generating excessive false positives on legitimate transactions.

Input Features:

- PCA-transformed features V1 through V28
- Transaction Time
- Transaction Amount.

Output Label:

- 0 for a normal transaction
- 1 for a fraudulent transaction

1.1 Project Goals

- Train multiple machine learning models for binary fraud classification
- Address severe class imbalance using appropriate techniques
- Avoid models that default to predicting only the majority class
- Evaluate models using imbalanced-learning metrics:
 - Precision
 - Recall
 - F1-Score
 - AUPRC (Area Under Precision-Recall Curve)
 - ROC-AUC
- Compare all models and identify the best-performing approach

1.2 Team Members' Responsibilities:

- **Alyssa Tu** – Responsible for the Logistic Regression model. Loaded and preprocessed the data, applied SMOTE oversampling to handle class imbalance, trained the linear baseline classifier, and computed evaluation scores including Precision, Recall, AUPRC, and the Precision-Recall curve. Also authored the project progress report and assisted in writing the final report.

- **Humayra Rashid** – Responsible for the Random Forest model and also explored a different approach using a CNN model. Built the Random Forest classifier using SMOTE oversampling to handle the class imbalance, evaluated it with a classification report, confusion matrix, AUPRC, and feature importance analysis. Also created a CNN model with class weights to test a deep learning approach for credit card fraud detection and compared its performance with that of the Random Forest model. In addition, assisted in writing the Final Project Report.
- **John Lopez** – Responsible for the XGBoost model. Trained a gradient-boosted tree classifier using the `scale_pos_weight` parameter for native class imbalance handling, produced Precision, Recall, F1-Score, AUPRC, and ROC-AUC metrics, and contributed a feature importance chart. In addition, worked on the slides.
- **Brian Andrade** – Responsible for the ANN (Artificial Neural Network) model using Backpropagation, Gradient Descent Optimization, and Grid Search Hyperparameter Tuning.
- **Slok Patel** – Responsible for model comparison and evaluation. Will aggregate Precision, Recall, and AUPRC results from all four models and select the best-performing approach.

2. Dataset Description

We use the publicly available ULB Credit Card Fraud Detection dataset. It contains credit card transactions made by European cardholders over two consecutive days in September 2013. It has a total of 284,807 transactions, of which only 492 are fraudulent, roughly 0.172% of all records. This extreme class imbalance is the defining challenge of the project. The dataset has no missing values, and all features are numeric, so no filling in missing values or heavy cleaning is required before modeling.

2.1 Features

- **V1–V28:** PCA-transformed numerical features (anonymized)
- **Time:** Seconds elapsed since first transaction
- **Amount:** Transaction value
- **Class:** Target label (0 = normal, 1 = fraud)

2.2 Dataset Split

Training data: 284,807 transactions with all features and the Class label. Test data: held-out 20% split used only for final evaluation.

All models use the same 80/20 stratified train-test split:

- 80% of transactions are used for training.
- 20% are held out for testing and final evaluation.
- `stratify=y` is used to preserve the fraud-to-normal ratio in both splits.
- `random_state=42` is used consistently across all models for reproducibility.

Because fraud accounts for less than 0.2% of transactions, accuracy is not a meaningful metric. A model that always predicts "normal" would achieve 99.8% accuracy while catching zero fraud cases. All models are therefore evaluated using Precision, Recall, F1-Score, and AUPRC instead.

3. Methods, Algorithms, and Tools

We implemented models using Python with the following libraries:

- pandas and numpy
- scikit-learn
- imbalanced-learn
- xgboost
- tensorflow / keras
- matplotlib

3.1 Logistic Regression (Baseline Model)

Logistic Regression served as the linear baseline model for this project. The model estimates the probability that a transaction is fraudulent by combining input features using a weighted sum, then applies a sigmoid activation function to produce a probability between 0 and 1. To address the severe class imbalance, SMOTE was applied only to the training data to generate synthetic fraud samples, balancing the classes without introducing data leakage into the test set.

The model achieved a recall of 0.92, indicating it correctly identified 92% of actual fraud cases. However, the precision was low, indicating many legitimate transactions were incorrectly flagged as fraud. Overall, the model achieved an AUPRC of 0.7703, which served as the main evaluation metric for comparing all models. This trade-off is expected in highly imbalanced datasets and establishes a strong baseline for evaluating more advanced models.

Key Configuration:

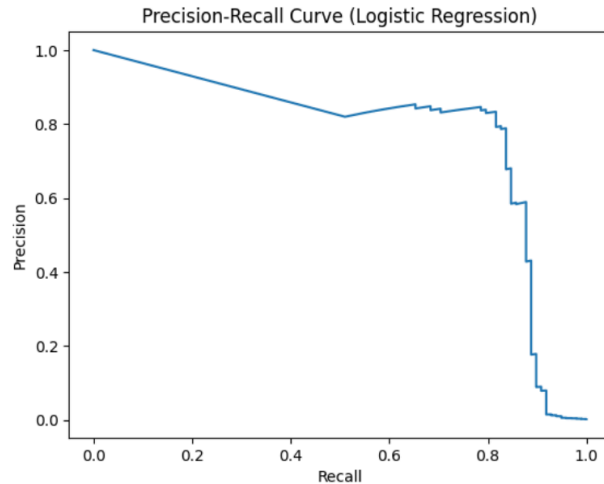
- max_iter = 1000 to prevent convergence warnings
- SMOTE was applied only to the training data to avoid data leakage

Results:

- Precision: 0.06
- Recall: 0.92
- AUPRC: 0.77

	precision	recall	f1-score	support
0	1.00	0.97	0.99	56864
1	0.06	0.92	0.11	98
accuracy			0.97	56962
macro avg	0.53	0.95	0.55	56962
weighted avg	1.00	0.97	0.99	56962

AUPRC: 0.7703412904668423



3.2 Random Forest Model (+ CNN Model - not fully included in final comparison)

Random Forest is an ensemble method that trains many decision trees and combines their predictions by majority vote. I used this model because it can handle non-linear relationships better than a single linear model and is more stable than using a single decision tree. Since the credit card fraud dataset is highly imbalanced, I applied SMOTE only to the training data so the model could learn more from the fraud class without changing the test data.

To explore a different approach, a CNN model was used. The CNN was used to test whether a deep learning approach could detect fraud patterns from the transaction features. Instead of using SMOTE with the CNN, class weights were used to give the model greater weight to the fraud class during training. Both models were evaluated using the classification report, confusion matrix, and AUPRC, so the results focused more on fraud detection than on accuracy alone.

Results:

Random Forest:

- Precision: 0.60
- Recall: 0.88
- AUPRC: 0.81

	precision	recall	f1-score	support
0	1.00	1.00	1.00	56864
1	0.60	0.88	0.71	98
accuracy			1.00	56962
macro avg	0.80	0.94	0.86	56962
weighted avg	1.00	1.00	1.00	56962

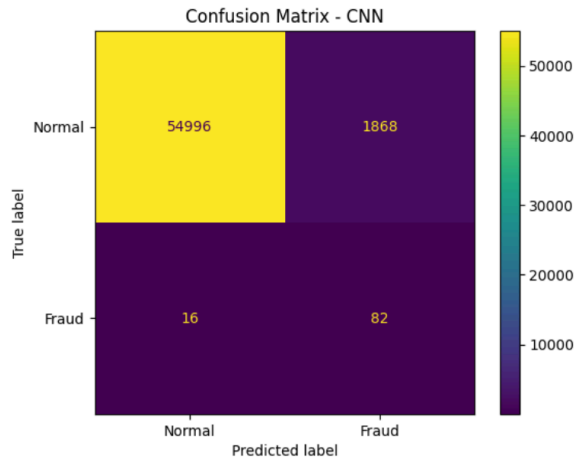
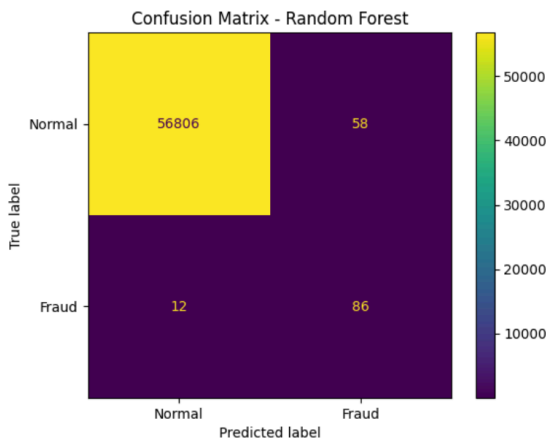
The Random Forest model has a strong recall, detecting 88% of actual fraud cases. The precision is also much higher than that of the CNN, meaning fewer normal transactions are incorrectly classified as fraud. Overall, Random Forest performs better for this fraud detection dataset and gives a strong result for comparison.

CNN:

- Precision: 0.04
- Recall: 0.84

	precision	recall	f1-score	support
0	1.00	0.97	0.98	56864
1	0.04	0.84	0.08	98
accuracy			0.97	56962
macro avg	0.52	0.90	0.53	56962
weighted avg	1.00	0.97	0.98	56962

- AUPRC: 0.70



The CNN model also has a high recall, meaning it detects 84% of actual fraud cases. However, the low precision indicates that many legitimate transactions are incorrectly classified as fraudulent. This means the CNN is good at detecting fraud, but it generates more false alarms than Random Forest.

3.3 XGBoost Model

XGBoost is a gradient boosted tree algorithm that builds an ensemble of decision trees sequentially. Each new tree is trained to correct the prediction errors of all previous trees, making the model progressively more accurate with each iteration. Unlike Random Forest, which trains trees independently, XGBoost uses gradient descent to minimize a loss function across all trees, making it more precise on difficult classification problems like fraud detection.

To handle the severe class imbalance, the `scale_pos_weight` parameter was set to the ratio of normal to fraud samples in the training data, approximately 577. This tells XGBoost to penalize missed fraud predictions 577 times more heavily than missed normal predictions, which is the recommended native approach for imbalanced data in XGBoost. SMOTE was not used because `scale_pos_weight` adjusts the loss function directly, which is cleaner and avoids introducing synthetic noise into the training data.

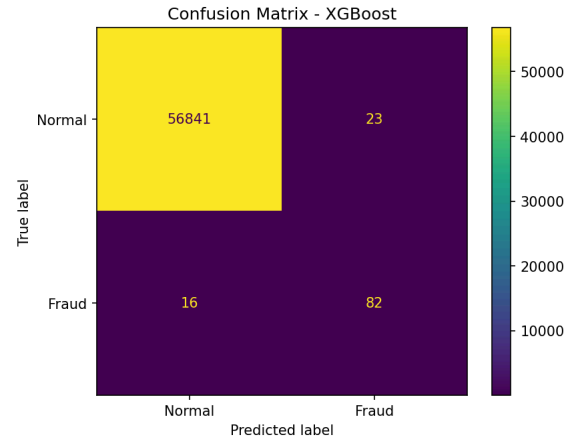
Key Configuration

- `n_estimators = 100`, number of boosted trees in the ensemble
- `max_depth = 6`, the maximum depth of each tree
- `learning_rate = 0.1`, step size for gradient descent optimization
- `scale_pos_weight = 577`, handles class imbalance natively without SMOTE

- `eval_metric = aucpr`, optimizes for precision, recall, and AUC during training.

Results:

- Precision: 0.7810
- Recall: 0.8367
- F1 Score: 0.8079
- AUPRC: 0.8650
- ROC AUC: 0.9747



3.4 ANN – Artificial Neural Network

The Artificial Neural Network model was built using scikit-learn's `MLPClassifier` with a Pipeline to ensure stable numerical scaling before each forward pass. The network learns through backpropagation, where prediction errors are propagated backward through the network layers and used to update each weight via gradient descent using the Adam optimizer. Adam is an adaptive learning rate optimizer that adjusts the step size for each weight individually, which makes it faster and more stable than standard gradient descent on large datasets.

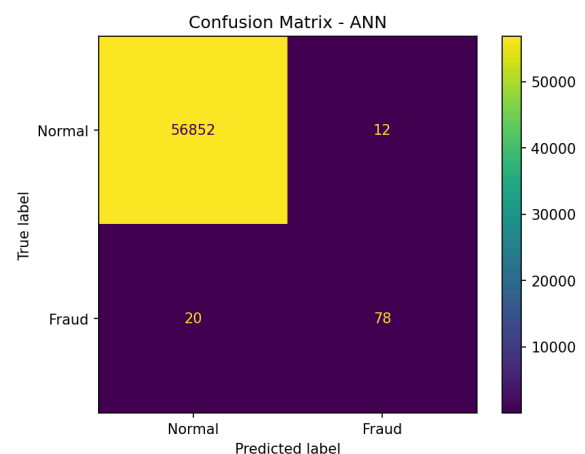
To find the best architecture, Grid Search with Stratified 3 Fold Cross Validation was used to search over hidden layer sizes and learning rate values. Stratified folds were used to ensure each fold preserved the fraud-to-normal ratio. Class weights were set to balance the classes, so the model automatically penalized missed fraud predictions in proportion to the class imbalance, without requiring SMOTE.

Key Configuration

- `solver = adam`, uses backpropagation with adaptive gradient descent, activation = ReLU, activation function for hidden layers.
- best `hidden_layer_sizes` found by Grid Search.
- best `learning_rate_init` found by Grid Search.
- `class_weight = balanced`, handles class imbalance natively.
- Cross Validation = Stratified 3-Fold.

Results:

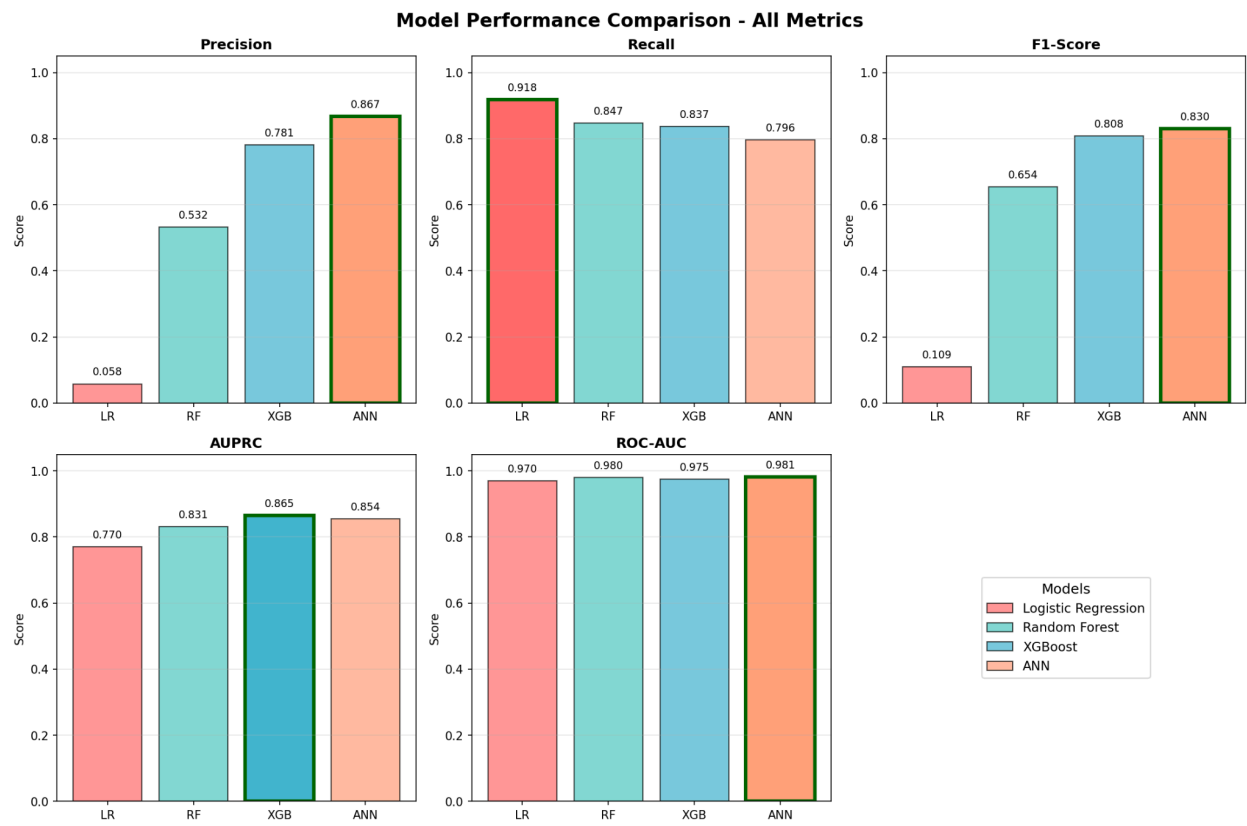
- Precision: 0.8667
- Recall: 0.7959



- F1 Score: 0.8298
- AUPRC: 0.8543
- ROC AUC: 0.9812

4. Overall Model Comparison

All four models were evaluated on the same held-out test set using Precision, Recall, F1 Score, AUPRC, and ROC AUC. Because fraud accounts for less than 0.2% of all transactions, AUPRC is the most meaningful metric for this comparison. AUPRC focuses specifically on the minority class and is not inflated by the large number of correct normal transaction predictions, unlike ROC AUC.



The table below summarizes the results across all four models.

Model	Precision	Recall	F1 Score	AUPRC	ROC AUC
Logistic Regression	0.0581	0.9184	0.1094	0.7703	0.9698

Random Forest	0.5321	0.8469	0.6535	0.8307	0.9798
XGBoost	0.7810	0.8367	0.8079	0.8650	0.9747
ANN	0.8667	0.7959	0.8298	0.8543	0.9812

XGBoost achieved the highest AUPRC at 0.8650, making it the best overall model for this fraud detection task. It also produced the best balance between precision and recall, catching 83.67% of all fraud cases while being correct 78.10% of the time it raised an alert. Logistic Regression had the highest recall at 91.84%, but its precision of 5.81% indicates it incorrectly flagged many normal transactions as fraud. The ANN achieved the highest precision and ROC AUC. Still, its AUPRC was slightly lower than XGBoost's, making XGBoost the stronger choice for a deployment scenario where minimizing both false negatives and false positives matters.

5. Conclusion and Future Work

This project demonstrated that machine learning models can effectively detect credit card fraud in a severely imbalanced dataset when appropriate evaluation metrics and class imbalance strategies are used. Four models were trained and compared: Logistic Regression, Random Forest, XGBoost, and an Artificial Neural Network. Each model used a different technique to handle the class imbalance, including SMOTE oversampling, `scale_pos_weight`, and class weights.

XGBoost was identified as the best performing model based on AUPRC, the most reliable metric for imbalanced classification. It achieved an AUPRC of 0.8650 and a ROC AUC of 0.9747 while maintaining strong precision and recall. A key finding from this project is that accuracy is a misleading metric for fraud detection. A model that predicts every transaction as normal achieves over 99% accuracy but catches zero fraud. Metrics like AUPRC and F1 Score tell the real story.

For future work, additional techniques such as cost sensitive learning, ensemble stacking, or threshold tuning could be explored to improve precision and recall. Real-time deployment would also require retraining on more recent transaction data to account for evolving fraud patterns. Also, the CNN model was trained but not fully included in the final comparison due to differences in the evaluation setup and limited time to test it using the same testing setup as the other models.

6. References & Resources

- ULB Credit Card Fraud Detection Dataset:
<https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud>

- Credit-Card-Fraud-Detection-Machine-Learning-Project Github Link:
<https://github.com/brinstussymethod/Credit-Card-Fraud-Detection-Machine-Learning-Project-.git>